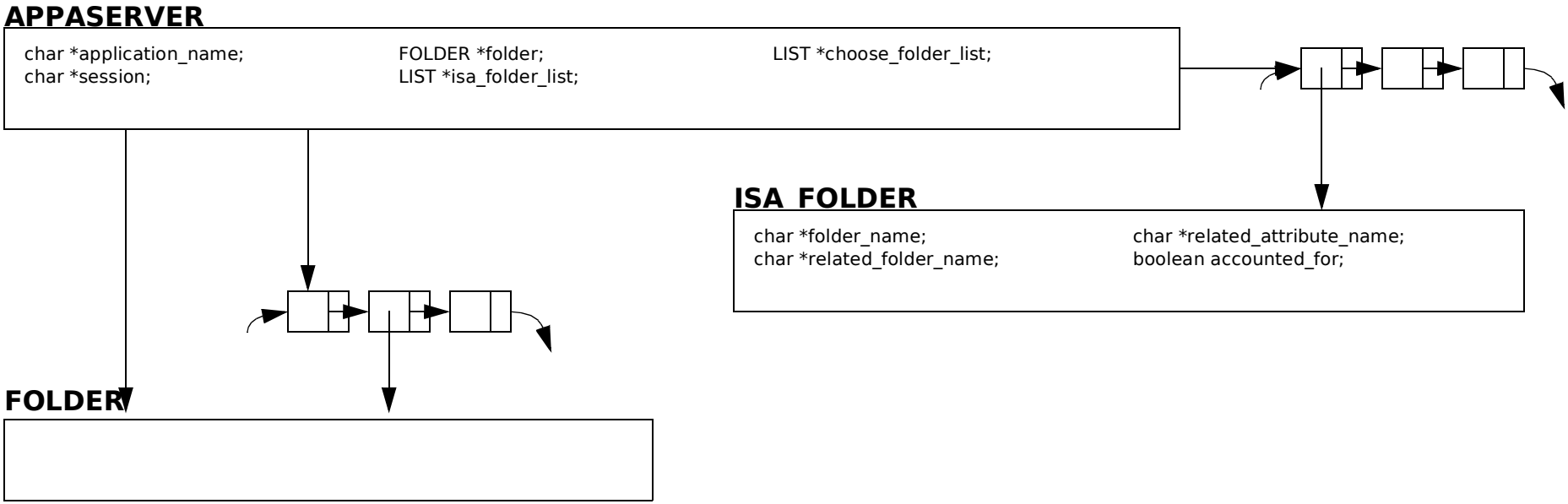
APPASERVER Data Structure

August 09, 2021



PROMPT_EDIT_FORM

```
/* Usage */
PROMPT_EDIT_FORM *
    prompt_edit_form_fetch(
        char *application_name,
        char *login_name,
        char *session,
        char *folder_name,
        char *role_name,
        char *state,
        char *target_frame,
        DICTIONARY *post_dictionary );

/* Input */
char *application_name;
char *login_name;
char *session;
char *folder_name;
char *role_name;
char *state;
char *target_frame;

/* Process */
ROLE *role =
    role_fetch(
        role_name,
        1 /* fetch_attribute_exclude_list */ );

FOLDER *folder =
    folder_fetch(
        folder_name,
        role_exclude_lookup_attribute_name_list(
            this->role->attribute_exclude_list ),
        /* Also sets primary_key_list */
        1 /* fetch_folder_attribute_list */,
        1 /* fetch_relation_mto1_isa_list */,
        1 /* fetch_relation_non_isa_list */,
        0 /* not fetch_process */,
        0 /* not fetch_row_level_restriction */ );

ROLE_FOLDER *role_folder =
    role_folder_fetch(
        role_name,
        folder_name );

DICTIONARY_APPASERVER *dictionary_appaserver =
    dictionary_appaserver_new(
        post_dictionary,
        application_name,
        folder->folder_attribute_isa_list,
        (LIST *)0 /* operation_name_list */ );

boolean prompt_edit_form_forbid(
    role->update_yn,
    role->lookup_yn );
```

FOLDER

ROLE

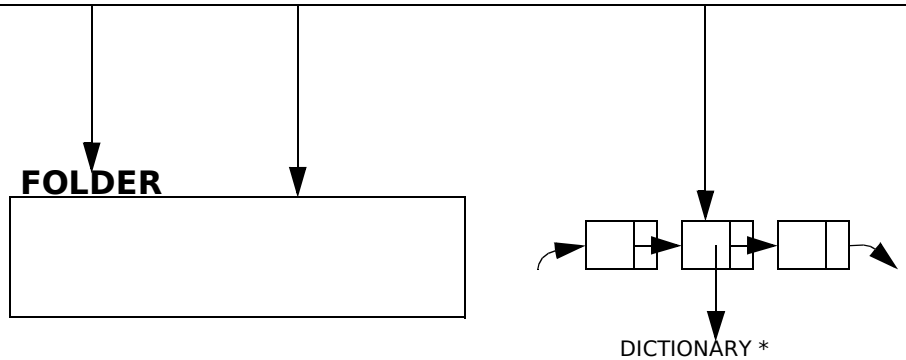
ROLE_FOLDER

DICTIONARY_APPASERVER

```
DICTIONARY *preprompt_dictionary;
```

RELATED_FOLDER

```
/* Input */
FOLDER *folder;
FOLDER *one2m_related_folder;
char *related_attribute_name;
boolean relation_type_isa;
int pair_1tom_order;
boolean null_1tom_upon_delete;
boolean ignore_output;
char *hint_message;
boolean copy_common_attributes;
boolean prompt_mto1_recursive;
boolean omit_1tom_detail;
char *join_where_clause;
boolean drop_down_multi_select;
boolean automatic_preselection;

/* Process */
LIST *foreign_attribute_name_list;
LIST *folder_foreign_attribute_name_list;
LIST *parent_foreign_attribute_name_list;
LIST *primary_data_list;
LIST *common_non_primary_key_list;
LIST *automatic_preselection_dictionary_list;
```



ROW_SECURITY (1)

```
/* Usage */
ROW_SECURITY *row_security_simple_new(
    query_dictionary,
    login_name,
    full_name_only,
    street_address_only,
    folder,
    appaserver_ignore_select_attribute_name_list() );

ROW_SECURITY *row_security_detail_new(
    application_name,
    where_attribute_data_list,
    folder,
    appaserver_ignore_select_attribute_name_list() );

/* Input */
char *state;
FOLDER *folder;
ROLE *role;
char *login_name;
char *full_name_only;
char *street_address_only;
DICTIONARY *preprompt_dictionary;
DICTIONARY *query_dictionary;
DICTIONARY *sort_dictionary;
LIST *role_update_list;
FOLDER *foreign_login_name_folder;
char *attribute_not_null_string;
FOLDER *attribute_not_null_folder;
LIST *ignore_select_attribute_name_list;

/* Process */
```

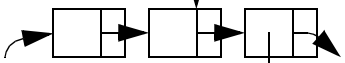
FOLDER

```
LIST *lookup_attribute_exclude_name_list;
LIST *update_attribute_exclude_name_list;
```

APPASERVER Data Structure

ROW_SECURITY (2)

```
/* Process */
LIST *row_security_role_update_list(
    void );
```

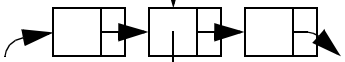


ROW_SECURITY_ROLE_UPDATE

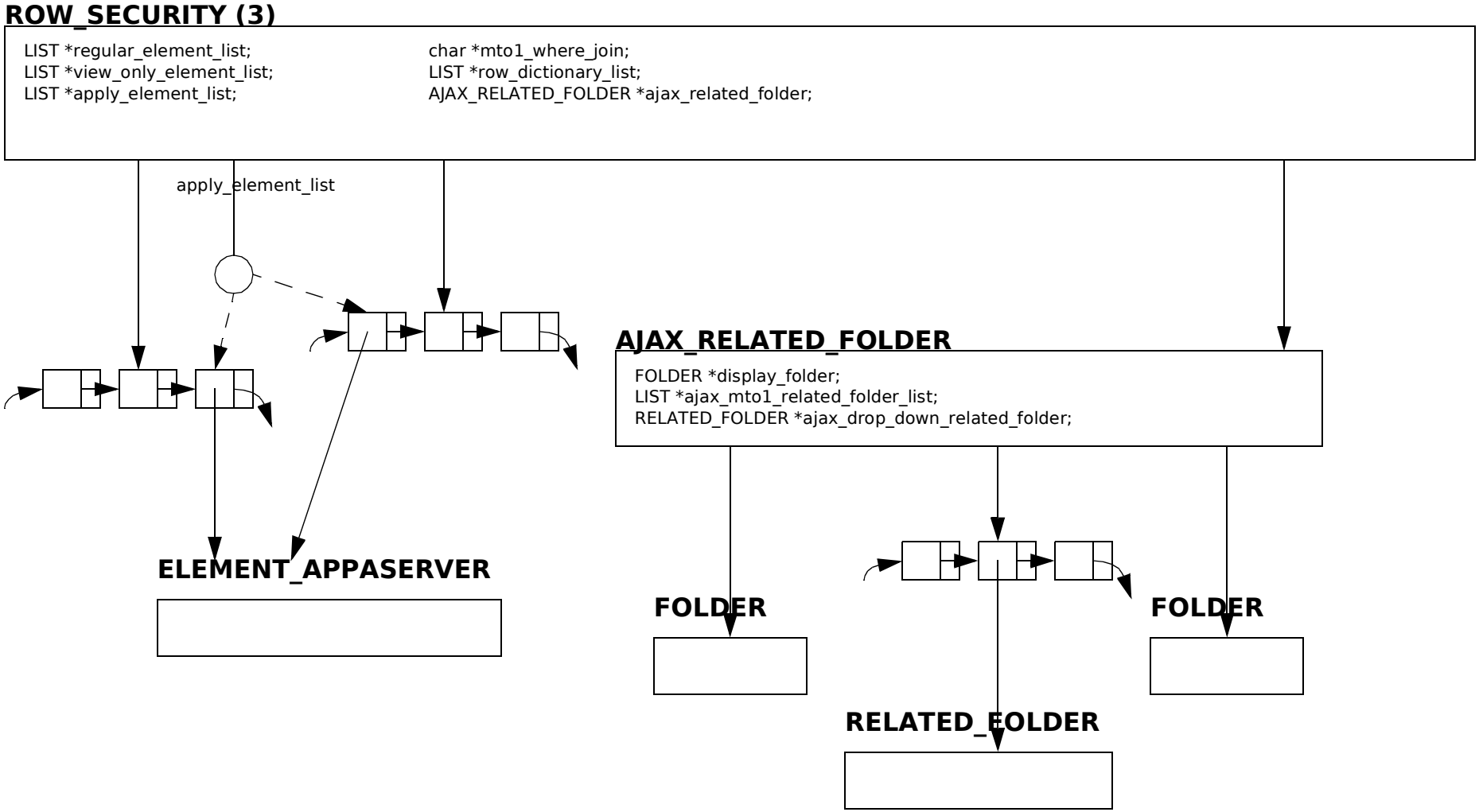
```
/* Usage */
ROW_SECURITY_ROLE_UPDATE *
    row_security_role_update_new(
        char *folder_name,
        char *attribute_not_null_string,
        ROLE *role );

FOLDER *role_update_folder;
char *attribute_not_null_string;
ROLE *role;

/* Process */
RELATED_FOLDER *row_security_role_update_related_seek(

boolean row_security_role_update_attribute_seek(
    attribute_name,
    row_security_role_update_list() );

boolean row_security_role_update_row_view_only(
    char *folder_name,
    FOLDER *role_update_folder,
    boolean role->override_row_restrictions,
    boolean attribute_not_null );

char *row_security_role_update_join(
```

FOLDER

```
char *folder_name;
LIST *one2m_recursive_related_folder_list;
```



RELATED_FOLDER


ROW_SECURITY (3)
LIST *regular_element_list;
LIST *view_only_element_list;
LIST *apply_element_list;
char *mto1_where_join;
LIST *row_dictionary_list;
AJAX_RELATED_FOLDER *ajax_related_folder;
apply_element_list
AJAX_RELATED_FOLDER
FOLDER *display_folder;
LIST *ajax_mto1_related_folder_list;
RELATED_FOLDER *ajax_drop_down_related_folder;
ELEMENT_APPASERVER
FOLDER
FOLDER
RELATED_FOLDER

ELEMENT_APPASERVER

```
char *name;                          boolean remember_keystrokes_ok;
enum element_type element_type;      int tab_index;
```

drop_down
push_button
radio_button
notepad
password
non_edit_text
linebreak
text_item
upload_filename
prompt_data
prompt_data_plus_hidden
reference_number
hidden
empty_column
blank
element_date
element_current_date

element_time
element_current_time
http_filename
timestamp
table_opening
table_closing
prompt
anchor

```
ELEMENT_DROP_DOWN *drop_down;
ELEMENT_PUSH_BUTTON *push_button;
ELEMENT_RADIO_BUTTON *radio_button;
ELEMENT_NOTEPAD *notepad;
ELEMENT_LINEBREAK *linebreak;
ELEMENT_TEXT_ITEM *text_item;
ELEMENT_PASSWORD *password;
ELEMENT_PROMPT_DATA *prompt_data;
ELEMENT_HIDDEN *hidden;
ELEMENT_UPLOAD_FILENAME *upload_filename;
ELEMENT_HTTP_FILENAME *http_filename;
ELEMENT_ANCHOR *anchor;
ELEMENT_EMPTY_COLUMN *empty_column;
ELEMENT_REFERENCE_NUMBER *reference_number;
ELEMENT_NON_EDIT_TEXT *non_edit_text;
```

APPASERVER Data Structure

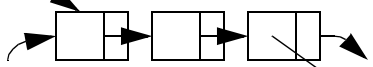
August 09, 2021

LOOKUP_BEFORE_DROP_DOWN

FOLDER *base_folder;
LIST *lookup_before_drop_down_folder_list;
boolean first_prelookup;

char *insert_pair_base_folder_name;
char *insert_folder_name;
boolean omit_lookup_before_drop_down;

enum lookup_before_drop_down_state lookup_before_drop_down_state;



lookup_not_participating
lookup_participating_not_root
lookup_skipped
lookup_participating_is_root_all_complete

FOLDER

char *folder_name;
boolean lookup_before_drop_down;
LIST *mto1_lookup_before_drop_down_related_folder_list;
LIST *pair_1tom_related_folder_list;

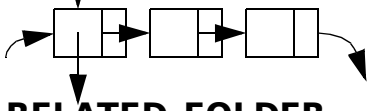
LOOKUP_BEFORE_DROP_DOWN_FOLDER

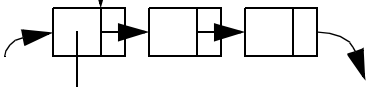
char *folder_name;
boolean fulfilled;

RELATED_FOLDER

PROMPT_RECURSIVE

```
/* Usage */
PROMPT_RECURSIVE *prompt_recursive_new(
    char *folder_name,
    LIST *mto1_related_folder_list );

/* Input */
char *folder_name;
LIST *mto1_related_folder_list;

/* Process */
LIST *prompt_recursive_folder_list(
    mto1_related_folder_list );

char *prompt_recursive_display(
    PROMPT_RECURSIVE *prompt_recursive );
```





RELATED_FOLDER

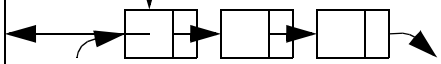
PROMPT_RECURSIVE_FOLDER

```
/* Usage */
LIST *prompt_recursive_folder_list(
    LIST *mto1_related_folder_list );

PROMPT_RECURSIVE_FOLDER *prompt_recursive_folder_new(
    FOLDER *related_folder->folder,
    boolean related_folder->drop_down_multi_select );

/* Input */
FOLDER *folder;
boolean drop_down_multi_select;

/* Process */
LIST *prompt_recursive_mto1_folder_list(
    related_folder->folder->folder_name,
    related_folder->folder->primary_key_list );
```

FOLDER

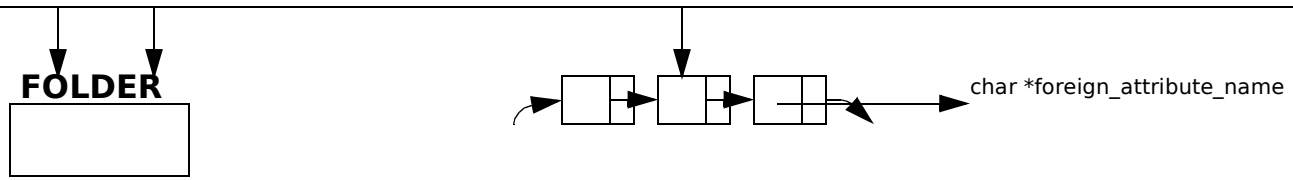
PROMPT_RECURSIVE_MTO1_FOLDER

```
/* Usage */
LIST *prompt_recursive_mto1_folder_list(
    char *folder_name,
    LIST *primary_key_list );

PROMPT_RECURSIVE_MTO1_FOLDER *
    prompt_recursive_mto1_folder_new(
        FOLDER *folder,
        int recursive_level );

FOLDER *folder;
int recursive_level;
```



RELATION

```
/* Input */
char *many_folder_name;
char *one_folder_name;
char *related_attribute_name;
int pair_1tom_order;
boolean omit_1tom_detail;
boolean prompt_mto1_recursive;
boolean relation_type_isa;
boolean copy_common_attributes;
boolean automatic_preselection;
boolean drop_down_multi_select;
boolean join_1tom_each_row;
boolean omit_lookup_before_drop_down;
boolean ajax_fill_drop_down;
char *hint_message;

/* Process */
FOLDER *many_folder;
FOLDER *one_folder;

char *relation_system_string(
    char *where,
    char *order );

LIST *relation_system_list(
    char *relation_system_string(),
    /* Setting both will set foreign_attribute_name_list */
    boolean fetch_folder,
    boolean fetch_attribute_list );

LIST *relation_one2m_pair_list(
    char *one_folder_name );

LIST *relation_mto1_non_isa_list(
    char *many_folder_name );

LIST *relation_one2m_list(]
    char *one_folder_name );

LIST *relation_one2m_recursive_list(
    LIST *relation_list /* in/out */,
    char *one_folder_name );

LIST *relation_mto1_non_isa_list(
    char *many_folder_name );

/* folder_attribute_list is set */
LIST *relation_mto1_isa_list(
    LIST *relation_list /* in/out */,
    char *many_folder_name );

boolean relation_is_primary_key_subset(
    LIST *foreign_attribute_name_list,
    LIST *mto1_primary_key_list );

LIST *relation_foreign_attribute_name_list(
    folder_attribute_primary_name_list(
        one_folder->
        folder_attribute_list )
        /* primary_foreign_attribute_name_list */,
    related_attribute_name,
    foreign_attribute_name_list(
        foreign_attribute_list(
            many_folder->folder_name,
            one_folder->folder_name ) );

char *relation_join_where_clause();
```



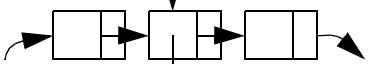
FOREIGN_ATTRIBUTE

```
/* Usage */
LIST *foreign_attribute_list(
    char *many_folder_name,
    char *one_folder_name );

/* Input */
char *folder;
char *related_folder;
char *related_attribute;
char *foreign_attribute;
int foreign_key_index;

/* Process */
LIST *foreign_attribute_name_list(
    foreign_attribute_list() );
```

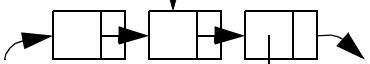
APPASERVER Data Structure

FOLDER (1)

```
/* Usage */
FOLDER *folder_fetch(
    security_sql_injection_escape(
        char *folder_name ),
    security_sql_injection_escape(
        char *role_name ),
    exclude_attribute_name_list,
    /* Also sets primary_key_list */
    boolean fetch_folder_attribute_list,
    boolean fetch_relation_mto1_non_isa_list,
    boolean fetch_relation_mto1_isa_list,
    boolean fetch_relation_one2m_list,
    boolean fetch_relation_one2m_recursive_list,
    boolean fetch_process,
    boolean fetch_role_folder_list,
    boolean fetch_row_level_restriction );

FOLDER *folder_primary_data_fetch(
    char *security_sql_injection_escape(
        folder_name ),
    DICTIONARY *preprompt_dictionary,
    PROCESS *populate_drop_down_process );

/* Input */
char *sql_injection_escape_older_name;
char *sql_injection_escape_role_name;
LIST *exclude_lookup_attribute_name_list;
char *folder_form;
int insert_rows_number;
boolean lookup_email_output;
char *notepad;
char *populate_drop_down_process_name;
char *post_change_process_name;
char *html_help_file_anchor;
char *post_change_javascript;
char *subschema;
boolean exclude_application_export;
boolean lookup_before_drop_down;
boolean no_initial_capital;
char *index_directory;
char *data_directory;
char *create_view_statement;
boolean appaserver;

/* Process */
PROCESS *populate_drop_down_process;
PROCESS *post_change_process;

char *folder_primary_where(
    char *security_sql_injection_escape(
        char *folder_name ) );

LIST *folder_attribute_list(
    this->folder_name,
    this->exclude_attribute_name_list );

LIST *primary_key_list =
    folder_attribute_primary_key_list(
        folder_attribute_list() );

LIST *folder_attribute_append_isa_list(
    list_copy( this->folder_attribute_list ),
    this->relation_mto1_isa_list );

LIST *role_folder_list(
    this->sql_injection_escape_role_name,
    this->sql_injection_escape_folder_name );

long int folder_row_count(
    char *folder_table_name() );

char *folder_row_level_restriction_string(
    this->sql_injection_escape_folder_name );

boolean folder_non_owner_view_only(
    folder_row_level_restriction_string() );

boolean folder_non_owner_forbid(
    folder_row_level_restriction_string() );
```



FOLDER_ATTRIBUTE



ROLE_FOLDER

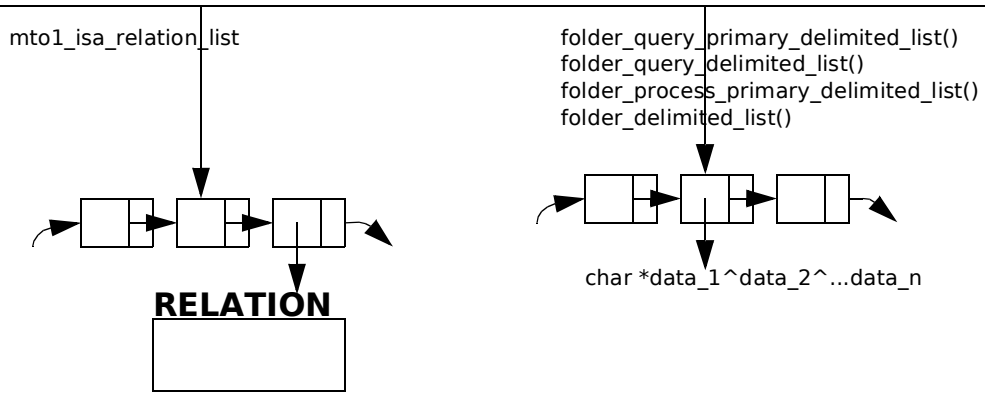
```
/* Usage */
LIST *role_folder_list(
    role_name,
    folder_name );

/* Input */
char *role_name;
char *folder_name;
char *permission;

/* Process */
boolean role_folder_insert(
    role_folder_list );

boolean role_folder_update(
    role_folder_list );

boolean role_folder_lookup(
    role_folder_list );
```

PROCESS

char *folder_row_level_restriction_string { row_level_non_owner_view_only, row_level_non_owner_forbid

FOLDER (2)

```
LIST *mto1_isa_relation_list =
    /* folder_attribute_list is set */
    relation_mto1_isa_list(
        LIST *this->mto1_isa_relation_list,
        char *folder_name );

LIST *folder_process_primary_delimited_list(
    PROCESS *populate_drop_down_process,
    DICTIONARY *preprompt_dictionary,
    char *login_name );

LIST *folder_query_primary_delimited_list(
    folder_table_name(),
    folder_attribute_list(),
    folder_attribute_primary_name_list(),
    DICTIONARY *preprompt_dictionary,
    char *login_name );

LIST *folder_query_delimited_list(
    char *folder_table_name(),
    LIST *folder_attribute_append_isa_list(),
    LIST *relation_mto1_isa_list(),
    DICTIONARY *query_dictionary );

LIST *folder_delimited_list(
    char *folder_table_name(),
    LIST *attribute_name_list,
    char *where_clause );

LIST *folder_dictionary_list(
    char *folder_table_name(),
    LIST *folder_attribute_append_isa_list(),
    LIST *relation_mto1_isa_list(),
    DICTIONARY *query_dictionary );

char *folder_table_name(
    char *environment_application_name(),
    char *folder_name );

boolean non_owner_forbid =
    folder_row_level_restrictions_non_owner_forbid(
        folder_name );

LIST *relation_mto1_non_isa_list(
    folder_name );

LIST *mto1_isa_recursive_related_folder_list;
LIST *mto1_copy_common_related_folder_list;
LIST *one2m_related_folder_list;
LIST *append_isa_attribute_list;
LIST *copy_common_attribute_list;
LIST *append_isa_copy_attribute_list;
LIST *primary_key_list;
LIST *append_isa_attribute_name_list;
LIST *copy_common_attribute_name_list;
LIST *append_isa_copy_attribute_name_list;
LIST *lookup_attribute_exclude_name_list;

LIST *attribute_float_list =
    attribute_extract_float_list(
        append_isa_attribute_list );
```



FOLDER_ATTRIBUTE

```
/* Usage */
LIST *folder_attribute_list(
    char *security_sql_injection_escape(
        folder_name ),
    LIST *exclude_attribute_name_list );

/* Input */
char *folder_name;
char *attribute_name;
int primary_key_index;
int display_order;
boolean omit_insert;
boolean omit_insert_prompt;
boolean omit_update;
boolean additional_unique_index;
boolean additional_index;
boolean insert_required;
boolean lookup_required;

ATTRIBUTE *attribute =
    attribute_fetch(
        attribute_name );

/* Process */
char *folder_attribute_primary_where(
    char *folder_name,
    char *attribute_name );

char *folder_attribute_where(
    char *sql_injection_escape_folder_name,
    LIST *exclude_attribute_name_list );

char *folder_attribute_system_string(
    folder_attribute_where() );

LIST *folder_attribute_system_list(
    folder_attribute_system_string() );

FOLDER_ATTRIBUTE *folder_attribute_parse(
    char *input );

LIST *folder_attribute_append_isa_list(
    LIST *folder_attribute_append_isa_list,
    LIST *relation_mto1_isa_list );

LIST *folder_attribute_primary_name_list(
    folder_attribute_list() );

boolean folder_attribute_exists(
    char *folder_name,
    char *attribute_name );
```

ATTRIBUTE

```
/* Usage */./
LIST *attribute_fetch(
    char *attribute_name );

/* Input */
char *attribute_name;
char *datatype_name;
int width;
int float_decimal_places;
char *hint_message;
char *post_change_javascript;
char *on_focus_javascript_function;
boolean lookup_histogram_output;
boolean lookup_time_chart_output;

/* Process */
static LIST *attribute_list( void );

LIST *attribute_seek(
    char *attribute_name,
    LIST *attribute_list() );
```

QUERY (1)

```
/* Usage */
QUERY *query_simple_new(
    DICTIONARY *query_dictionary,
    char *security_sql_injection_escape(
        folder_name ),
    char *security_sql_injection_escape(
        role_name ),
    char *security_sql_injection_escape(
        login_name ),
    LIST *dictionary_appaserver->
        ignore_select_attribute_name_list );

QUERY *query_edit_table_new(
    DICTIONARY *query_dictionary,
    char *security_sql_injection_escape(
        folder_name ),
    char *security_sql_injection_escape(
        role_name ),
    char *security_sql_injection_escape(
        login_name ),
    LIST *dictionary_appaserver->
        ignore_select_attribute_name_list,
    char *attribute_not_null_join,
    char *attribute_not_null_folder_name,
    DICTIONARY *sort_dictionary );

QUERY *query_detail_new(
    LIST *where_attribute_data_list,
    char *security_sql_injection_escape(
        folder_name ),
    char *security_sql_injection_escape(
        role_name ),
    char *security_sql_injection_escape(
        login_name ),
    LIST *dictionary_appaserver->
        ignore_select_attribute_name_list,
    char *attribute_not_null_join,
    char *attribute_not_null_folder_name );

QUERY *query_primary_delimited_new(
    DICTIONARY *preprompt_dictionary,
    char *folder_name,
    char *role_name,
    char *login_name );

QUERY *query_process_parameter_new(
    DICTIONARY *preprompt_dictionary,
    char *folder_name,
    char *role_name,
    char *login_name );

/* Input */
DICTIONARY *query_dictionary;
LIST *where_attribute_data_list;
char *sql_injection_escape_folder_name;
char *sql_injection_escape_role_name;
char *sql_injection_escape_login_name;
LIST *ignore_select_attribute_name_list;
char *attribute_not_null_join;
char *attribute_not_null_folder_name;
DICTIONARY *sort_dictionary;
```

```
enum
relational_operator {
    equals                  not_equal               not_equal_or_null
    less_than               less_than_equal_to      greater_than
    greater_than_equal_to   between                 begins
    contains                not_contains            query_or
    is_null                 not_null
```

QUERY (2)

```
/* Process */
int max_rows = QUERY_MAX_ROWS;

ROLE *role =
    role_fetch(
        role_name,
        1 /* fetch_attribute_exclude_list */ );

FOLDER *query_folder =
    folder_fetch(
        this->sql_injection_escape_folder_name,
        this->sql_injection_escape_role_name,
        exclude_lookup_attribute_name_list,
        /* Also sets primary_key_list */
        1 /* fetch_folder_attribute_list */,
        0 /* not fetch_relation_mto1_non_isa_list */,
        1 /* fetch_relation_mto1_isa_list */,
        0 /* not fetch_relation_one2m_list */,
        0 /* not fetch_relation_one2m_recursive_list */,
        1 /* fetch_process */,
        1 /* fetch_role_folder_list */,
        1 /* fetch_row_level_restriction */ );

SECURITY_ENTITY *security_entity(
    this->login_name,
    query_folder->non_owner_forbid,
    role->override_row_restrictions );

PROMPT_RECURSIVE *prompt_recursive =
    prompt_recursive_new(
        query_folder->folder_name,
        query_folder->mto1_related_folder_list );

ROLE_FOLDER *role_folder =
    role_folder_fetch(
        role_name,
        folder_name );

LIST *query_common_attribute_name_list();
```

PROMPT_RECURSIVE

ROLE_FOLDER

ROLE

```
/* Usage */
ROLE *role_fetch(
    char *role_name,
    boolean fetch_attribute_exclude_list );

/* Input */
char *role_name;
boolean folder_count;
boolean override_row_restrictions;
LIST *attribute_exclude_list;

/* Process */
char *role_primary_where(
    char *role_name );

LIST *role_exclude_lookup_attribute_name_list(
    attribute_exclude_list );

LIST *role_exclude_insert_attribute_name_list(
    attribute_exclude_list );

LIST *role_exclude_update_attribute_name_list(
    attribute_exclude_list );
```

SECURITY_ENTITY

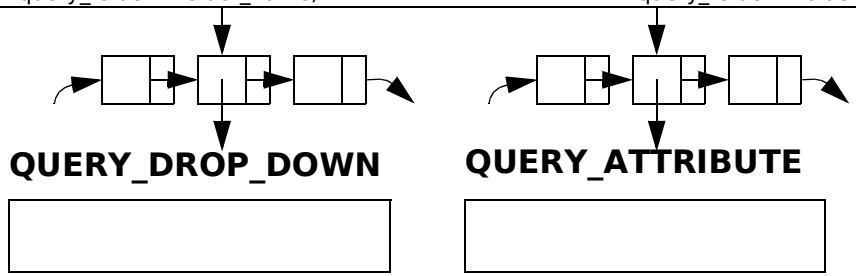
FOLDER

ROLE_ATTRIBUTE_EXCLUDE

```
char *role_name;
char *attribute_name;
char *permission;
```

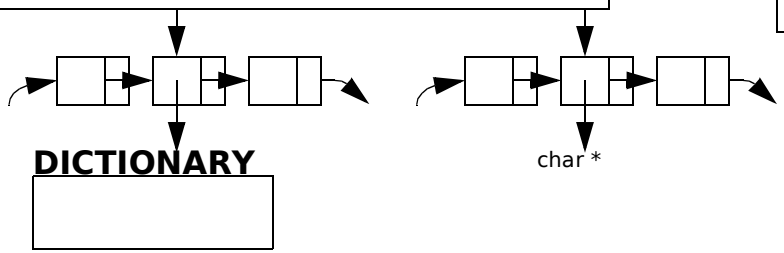
{ insert
update
lookup

```
/* Process */
LIST *exclude_attribute_name_list = list_new();

LIST *query_drop_down_list(
    exclude_attribute_name_list,
    query_folder,
    query_dictionary );

LIST *query_prompt_recursive_drop_down_list(
    exclude_attribute_name_list,
    query_folder->folder_name
        /* query_folder_name */,
    prompt_recursive->
        prompt_recursive_folder_list,
    query_dictionary );

LIST *query_attribute_list(
    query_folder->append_isa_attribute_list,
    query_dictionary,
    exclude_attribute_name_list );

char *query_drop_down_where(
    query_drop_down_list(),
    query_folder->folder_name,
    list_length(
        query_folder->
            mto1_isa_related_folder_list ) );

char *query_drop_down_data_where(
    char *query_folder_name,
    char *attribute_name,
    char *data );

char *query_attribute_where(
    query_attribute_list() );

char *query_join_where(
    query_folder->folder_name,
    query_folder->primary_key_list,
    query_folder->mto1_isa_related_folder_list,
    query->attribute_not_null_join );

char *query_related_join(
    query_folder->folder_name,
    query_folder->primary_key_list,
    isa_related_folder->folder->folder_name,
    isa_related_folder->
        foreign_attribute_name_list );

char *query_where(
    query_drop_down_where(),
    query_attribute_where(),
    query_join_where(),
    attribute_not_null_join );

char *query_display_where(
    char *query_where(),
    char *folder_name );

LIST *query_select_name_list(
    query_foler->append_isa_attribute_list,
    ignore_select_attribute_name_list,
    query_folder->
        lookup_attribute_exclude_name_list,
    common_attribute_name_list );

char *query_select_display(
    query_folder->folder_name,
    LIST *query_select_name_list(),
    int list_length(
        query_folder->
            mto1_isa_related_folder_list ),
    common_attribute_name_list );

char *query_from(
    query_folder->folder_name,
    query_folder->mto1_isa_related_folder_list,
    attribute_not_null_folder_name );

char *query_order(
    query_folder->folder_name,
    query_folder->primary_key_list,
    query_folder->append_isa_attribute_list,
    sort_dictionary );

QUERY_DATE_CONVERT *query_date_convert(
        query_entity->login_name,
        query_folder->append_isa_attribute_list );

QUERY *query_calloc(
    void );

LIST *query_where_attribute_name_list(
    LIST *foreign_attribute_name_list );

char *query_dictionary_operator_name(
    char *attribute_name,
    DICTIONARY *query_dictionary,
    int dictionary_offset,
    char *starting_label );

enum relational_operator query_relational_operator(
    char *query_dictionary_operator_name(),
    char *attribute_datatype );

char *query_sort_prefix_direction_attribute_index(
    DICTIONARY *sort_dictionary );

char *query_attribute_name_list_order(
    char *folder_name,
    LIST *primary_key_list,
    LIST *append_isa_attribute_list,
    char *descending_label );
```



QUERY (4)

```
LIST *query_dictionary_list(
    query_select_display(),
    query_select_name_list(),
    query_from(),
    query_where(),
    query_order(),
    max_rows,
    query_date_convert() );

LIST *query_record_list(
    query_select_display(),
    query_select_name_list(),
    query_from(),
    query_where(),
    query_order(),
    max_rows,
    query_date_convert() );
```



QUERY_DATE_CONVERT

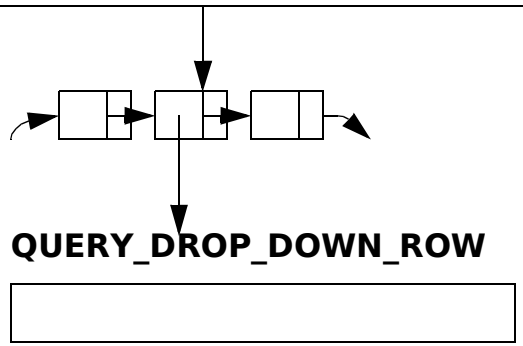
```
/* Usage */
QUERY_DATE_CONVERT *query_date_convert_new(
    char *login_name,
    LIST *primary_key_list,
    LIST *append_isa_attribute_list );

/* Process */
LIST *date_attribute_name_list =
    attribute_date_attribute_name_list(
        append_isa_attribute_list );

LIST *date_position_list =
    query_date_convert_position_list(
        primary_key_list,
        date_attribute_name_list );

enum date_convert_format database_date_format =
        date_convert_format_evaluate(
            application_database_date_format() );

enum date_convert_format user_date_format =
        date_convert_format_evaluate(
            date_convert_login_name_fetch(
                login_name )->
                    destination_format );
```

APPASERVER Data Structure

QUERY_DROP_DOWN

```
/* Usage */
LIST *query_drop_down_list(
    LIST *exclude_attribute_name_list /* in/out */,
    FOLDER *query_folder,
    DICTIONARY *query_dictionary );

QUERY_DROP_DOWN *query_drop_down_new(
    LIST *exclude_attribute_name_list /* in/iout */,
    char *query_folder_name,
    char *one2m_folder_name,
    LIST *one2m_primary_key_list,
    LIST *mto1_foreign_attribute_name_list,
    LIST *one2m_attribute_list,
    DICTIONARY *query_dictionary );

/* Input */
char *query_folder_name;
char *one2m_folder_name;
LIST *one2m_primary_key_list;
LIST *mto1_foreign_attribute_name_list;
LIST *one2m_attribute_list;
```

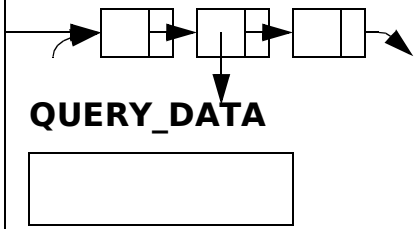
```
/* Process */
QUERY_DROP_DOWN *query_drop_down_fetch(
    char *query_folder_name,
    char *one2m_folder_name,
    LIST *one2m_primary_key_list,
    LIST *mto1_foreign_attribute_name_list );

LIST *drop_down_row_list =
    query_drop_down_row_list(
        exclude_attribute_name_list,
        query_folder_name,
        one2m_folder_name,
        one2m_primary_key_list,
        mto1_foreign_attribute_name_list,
        one2m_attribute_list,
        query_dictionary,
        dictionary_attribute_name_list_highest_index(
            query_dictionary,
            one2m_primary_key_list ) );
```



APPASERVER Data Structure

QUERY_DROP_DOWN_ROW

```
/* Usage */
LIST *query_drop_down_row_list(
    LIST *exclude_attribute_name_list /* in/iout */,
    char *query_folder_name,
    char *one2m_folder_name,
    LIST *one2m_primary_key_list,
    LIST *mto1_foreign_attribute_name_list,
    LIST *one2m_attribute_list,
    DICTIONARY *query_dictionary,
    int highest_index );

QUERY_DROP_DOWN_ROW *query_drop_down_row_new(
    LIST *exclude_attribute_name_list /* in/iout */,
    char *query_folder_name,
    char *one2m_folder_name,
    LIST *one2m_primary_key_list,
    LIST *mto1_foreign_attribute_name_list,
    LIST *one2m_attribute_list,
    DICTIONARY *query_dictionary,
    int index );

QUERY_DROP_DOWN_ROW *query_drop_down_row_fetch(
    char *query_folder_name,
    char *one2m_folder_name,
    LIST *one2m_primary_key_list,
    LIST *mto1_foreign_attribute_name_list,
    LIST *one2m_attribute_list,
    int index );

/* Input */
char *query_folder_name;
char *one2m_folder_name;
LIST *one2m_primary_key_list;
LIST *mto1_foreign_attribute_name_list;
LIST *one2m_attribute_list;
int index;

/* Process */
QUERY_DROP_DOWN_ROW *query_drop_down_row_calloc(
    void );

char *key =
    query_key(
        LIST *mto1_foreign_attribute_name_list,
        int index );

char *operator_name =
    query_dictionary_operator_name(
        query_key(),
        query_dictionary,
        QUERY_RELATION_OPERATOR_STARTING_LABEL );

enum relational_operator relational_operator =
    query_relational_operator(
        query_dictionary_operator_name(),
        (char *)0 /* attribute_datatype */ );

char *data_list_string =
    query_data_list_string(
        query_dictionary,
        key );

LIST *data_string_list =
    query_data_string_list(
        data_list_string );

LIST *query_data_list =
    query_drop_down_query_data_list(
        one2m_primary_key_list,
        mto1_foreign_attribute_name_list,
        one2m_attribute_list,
        data_string_list );
```



QUERY_ATTRIBUTE

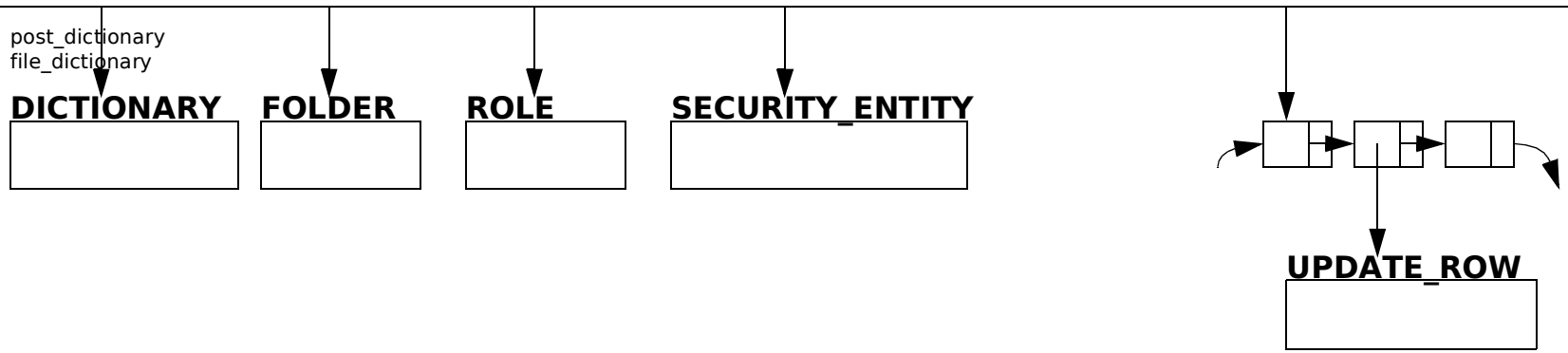
```
/* Usage */
LIST *query_attribute_list(
        LIST *append_isa_attribute_list,
        DICTIONARY *query_dictionary,
        LIST *exclude_attribute_name_list );

QUERY_ATTRIBUTE *query_attribute_new(
        char *attribute_name,
        char *folder_name,
        char *datatype,
        enum relation_operator relation_operator;
        QUERY_DATA *from_data;
        QUERY_DATA *to_data;
        int primary_key_index );

/* Input */
char *attribute_name;
char *folder_name;
char *datatype;
enum relational_operator relational_operator;
QUERY_DATA *from_data;
QUERY_DATA *to_data;
int primary_key_index;

/* Process */
char *query_attribute_list_display(
        char *subquery_folder_name,
        LIST *query_attribute_list );

boolean query_attribute_exists(
        LIST *query_attribute_list,
        char *attribute_name );

boolean query_attribute_date_time_between(
        QUERY_ATTRIBUTE **date_between_attribute,
        QUERY_ATTRIBUTE **time_between_attribute,
        LIST *query_attribute_list() );

char *query_attribute_between_date_time_where(
        char *date_attribute_name,
        char *time_attribute_date,
        QUERY_DATA *date_from_data,
        QUERY_DATA *time_from_data,
        QUERY_DATA *date_to_data,
        QUERY_DATA *time_to_data,
        char *folder_name );

boolean query_attribute_is_date(
        char *datatype );

boolean query_attribute_is_time(
        char *datatype );

boolean query_attribute_is_number(
        char *datatype );

boolean query_attribute_is_date_time(
        char *datatype );
```

QUERY_DATA

```
/* Usage */
LIST *query_drop_down_query_data_list(
    LIST *one2m_primary_key_list,
    LIST *mto1_foreign_attribute_name_list,
    LIST *one2m_attribute_list,
    LIST *data_string_list );

QUERY_DATA *query_data_new(
    char *attribute_name,
    char *attribute_datatype,
    char *security_sql_injection_escape(
        security_replace_special_characters(
            string_trim_number_characters(
                char *data,
                char *attribute_datatype ) ) ) );

/* Input */
char *attribute_name;
char *attribute_datatype;
char *escaped_replaced_data;

/* Process */
char *escaped_replaced_data =
    query_data_convert_date_international(
        char *attribute_datatype,
        char *escaped_replaced_data );
```

UPDATE

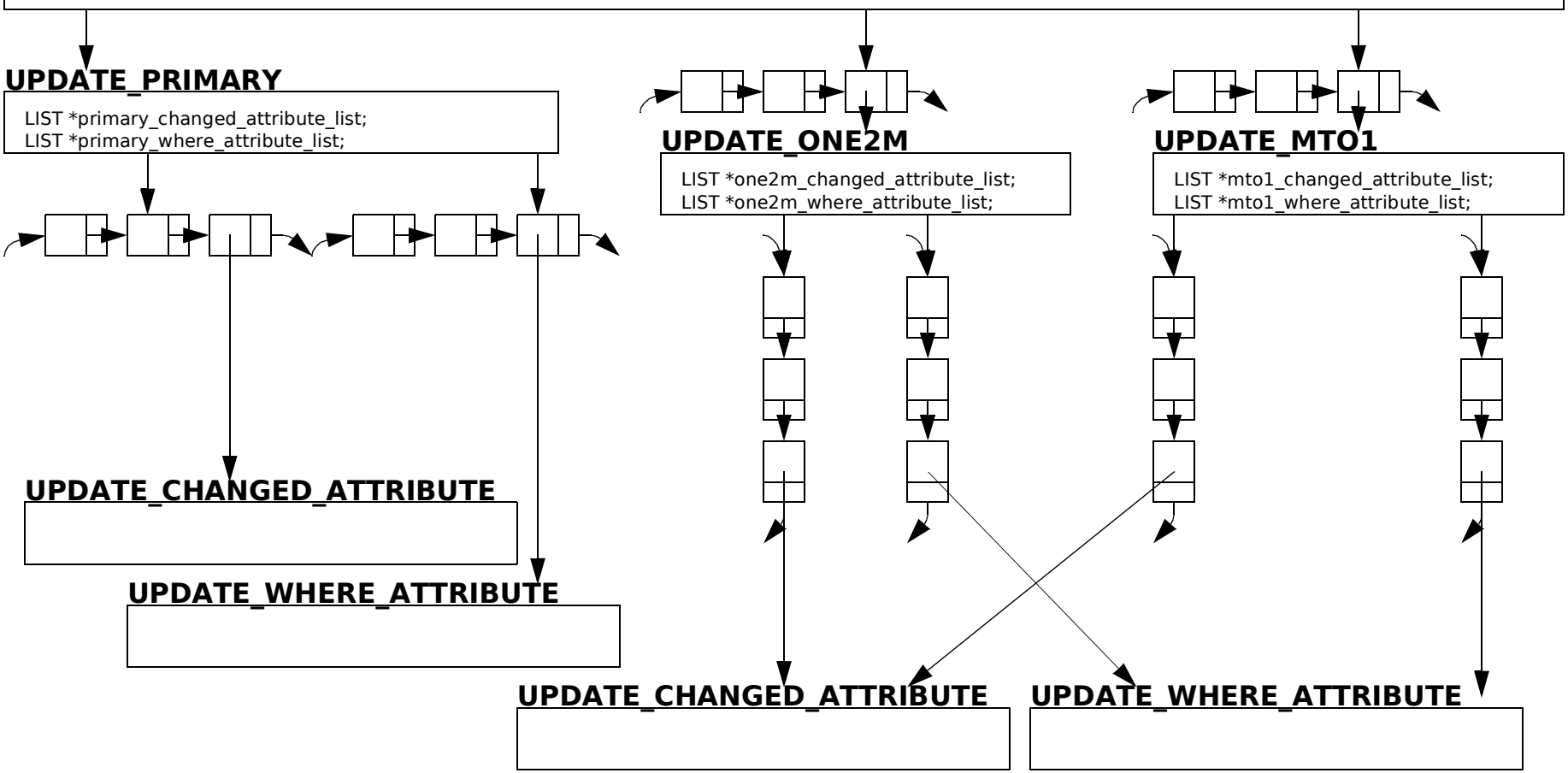
```
/* Usage */
UPDATE *update(
    /* Sets preupdate_$attribute_name for trigger */
    DICTIONARY *post_dictionary /* in/out */,
    DICTIONARY *file_dictionary,
    char *login_name,
    char *role_name,
    char *folder_name );

/* Input */
DICTIONARY *post_dictionary;
DICTIONARY *file_dictionary;
char *login_name;
char *role_name;
char *folder_name;

/* Process */
ROLE *role =
    role_fetch(
        role_name,
        1 /* fetch_attribute_exclude_list */ );

FOLDER *folder =
    folder_fetch(
        folder_name,
        role_name,
        role_exclude_update_attribute_name_list(
            role->attribute_exclude_list ),
        /* Also sets primary_key_list */
        1 /* fetch_folder_attribute_list */,
        0 /* not fetch_relation_mto1_non_isa_list */,
        1 /* fetch_relation_mto1_isa_list */,
        0 /* not fetch_relation_one2m_list */,
        1 /* fetch_relation_one2m_recursive_list */,
        1 /* fetch_process */,
        1 /* fetch_role_folder_list */,
        1 /* fetch_row_level_restriction */ );

if ( !role_folder_update(
        folder->role_folder_list ) )
{
    return (UPDATE *)0;
}

SECURITY_ENTITY *security_entity(
    login_name,
    folder->non_owner_forbid,
    role->override_row_restrictions );

LIST *update_row_list(
    DICTIONARY *post_dictionary,
    DICTIONARY *file_dictionary,
    FOLDER *folder,
    SECURITY_ENTITY *security_entity );
```



APPASERVER Data Structure

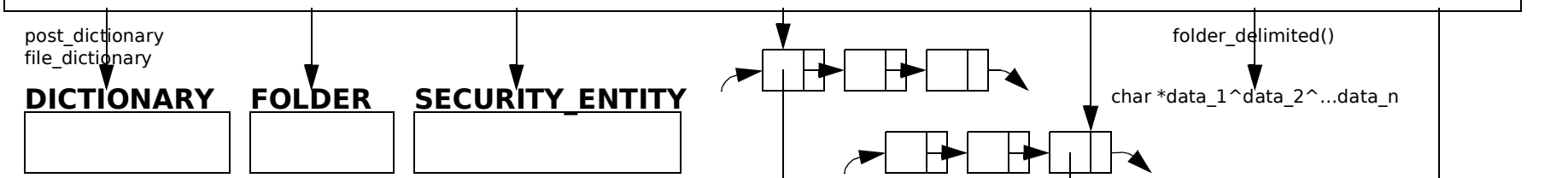
UPDATE_ROW

```
/* Usage */
UPDATE_ROW *update_row(
    DICTIONARY *post_dictionary,
    DICTIONARY *file_dictionary,
    FOLDER *folder,
    SECURITY_ENTITY *security_entity,
    int row );

/* Input */
int row;

/* Process */
UPDATE_PRIMARY *update_primary(
    post_dictionary,
    file_dictionary,
    folder,
    security_entity,
    int row );

LIST *update_one2m_list(
    update_primary->
        primary_key_changed_attribute_list,
    update_primary->
        primary_where_attribute_list,
    folder->relation_one2m_recursive_list );

LIST *update_mto1_list(
    update_primary->
        primary_key_changed_attribute_list,
    update_primary->
        primary_where_attribute_list,
    folder->relation_mto1_isa_list );

int update_row_cell_count(
    list_length(
        update_primary->
            primary_changed_attribute_list )
                /* primary_cell_count */,
    update_folder_one2m_list(),
    update_folder_mto1_list() );
```



UPDATE_PRIMARY

```
/* Usage */
UPDATE_PRIMARY *update_primary(
    /* Sets preupdate_$attribute_name for trigger */
    DICTIONARY *post_dictionary,
    DICTIONARY *file_dictionary,
    FOLDER *folder,
    SECURITY_ENTITY *security_entity,
    int row );

/* Process */
LIST *update_primary_changed_attribute_list(
    /* Sets preupdate_$attribute_name for trigger */
    post_dictionary,
    file_dictionary,
    folder->folder_attribute_list,
    row );

LIST *update_primary_where_attribute_list(
    file_dictionary,
    folder->primary_key_list,
    row );

char *security_entity_where_clause(
    update_where_clause(
        update_primary_where_attribute_list()
            /* where_attribute_list */ ),
    security_entity );

char *folder_delimited(
    folder_table_name(
        environment_application_name(),
        folder->folder_name ),
    folder->primary_key_list,
    security_entity_where_clause()
        /* primary_where_clause */ );

LIST *update_primary_key_changed_attribute_list(
    update_primary_changed_attribute_list() );

char *update_primary_set_clause(
    update_primary_changed_attribute_list() );
```



UPDATE_CHANGED_ATTRIBUTE

```
/* Usage */
UPDATE_CHANGED_ATTRIBUTE *
    update_changed_attribute(
        DICTIONARY *post_dictionary,
        DICTIONARY *file_dictionary,
        FOLDER_ATTRIBUTE *folder_attribute,
        int row );

/* Process */
UPDATE_CHANGED_ATTRIBUTE_DATA *
    update_changed_attribute_data;
```

UPDATE_CHANGED_ATTRIBUTE

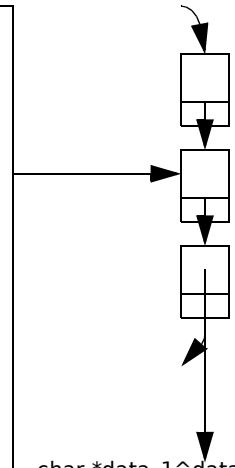
UPDATE_WHERE_ATTRIBUTE

```
/* Usage */
UPDATE_WHERE_ATTRIBUTE *
    update_where_attribute(
        DICTIONARY *file_dictionary,
        char *primary_key,
        int row );

LIST *
    update_foreign_where_attribute_list(
        LIST *primary_key_list,
        LIST *foreign_key_list );

/* Process */
char *attribute_name;
char *data;
char *foreign_attribute_name;

char *update_where_clause(
    LIST *where_attribute_list );
```

APPASERVER Data Structure

UPDATE_ONE2M

```
/* Usage */
UPDATE_ONE2M *update_one2m(
    LIST *primary_key_changed_attribute_list,
    LIST *primary_where_attribute_list,
    RELATION *relation_one2m );

/* Input */
LIST *primary_key_changed_attribute_list;
LIST *primary_where_attribute_list;
RELATION *relation_one2m;

/* Process */
LIST *update_one2m_changed_attribute_list(
    LIST *primary_key_changed_attribute_list,
    char *relation_one2m->many_folder->folder_name
        /* many_folder_name */,
    LIST *relation_one2m->foreign_attribute_name_list );

LIST *update_one2m_where_attribute_list(
    LIST *primary_where_attribute_list,
    LIST *relation_one2m->foreign_key_list );

LIST *folder_delimited_list(
    folder_table_name(
        environment_application_name(),
        relation_one2m->
            many_folder->
            folder_name ),
    LIST *relation_one2m->
        many_folder->
        primary_key_list,
    update_where_clause(
        update_one2m_where_attribute_list() ) );
```



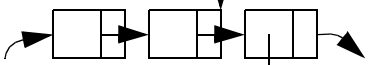
char *data_1^data_2^...data_n

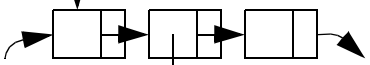
primary_key_changed_attribute_list

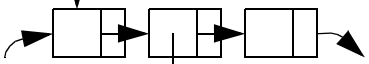
primary_where_attribute_list

RELATION *relation_one2m

one2m_changed_attribute_list







RELATION

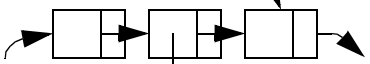
```
FOLDER *many_folder;
LIST *relation_foreign_attribute_name_list();
```

one2m_where_attribute_list

UPDATE_CHANGED_ATTRIBUTE

```
FOLDER_ATTRIBUTE *folder_attribute;
UPDATE_CHANGED_ATTRIBUTE_DATA *
        update_changed_attribute_data;
```

UPDATE_CHANGED_ATTRIBUTE



UPDATE_WHERE_ATTRIBUTE

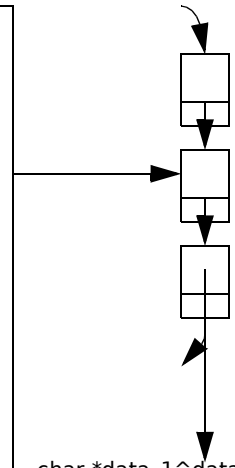
```
char *attribute_name;
char *data;
```

UPDATE_WHERE_ATTRIBUTE

APPASERVER Data Structure

UPDATE_MTO1

```
/* Usage */
UPDATE_MTO1 *update_mto1(
    LIST *primary_key_changed_attribute_list,
    LIST *primary_where_attribute_list,
    RELATION *relation_mto1 );

/* Input */
LIST *primary_key_changed_attribute_list;
LIST *primary_where_attribute_list;
RELATION *relation_mto1;

/* Process */
LIST *update_mto1_changed_attribute_list(
    LIST *primary_key_changed_attribute_list,
    char *relation_mto1->one_folder->folder_name
        /* one_folder_name */,
    LIST *relation_mto1->foreign_attribute_name_list );

LIST *update_mto1_where_attribute_list(
    LIST *primary_where_attribute_list,
    LIST *relation_mto1->foreign_key_list );

LIST *folder_delimited_list(
    folder_table_name(
        environment_application_name(),
        relation_mto1->
            one_folder->
            folder_name ),
    LIST *relation_mto1->
        one2m_folder->
        primary_key_list,
    update_where_clause(
        update_mto1_where_attribute_list() ) );
```



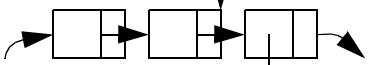
char *data_1^data_2^...data_n

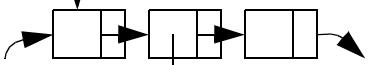
primary_key_changed_attribute_list

primary_where_attribute_list

RELATION *relation_mto1

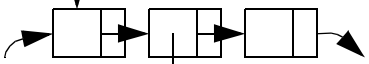
mto1_changed_attribute_list





RELATION

```
FOLDER *one_folder;
LIST *relation_foreign_attribute_name_list();
```

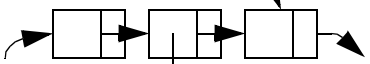


mto1_where_attribute_list

UPDATE_CHANGED_ATTRIBUTE

```
FOLDER_ATTRIBUTE *folder_attribute;
UPDATE_CHANGED_ATTRIBUTE_DATA *
        update_changed_attribute_data;
```

UPDATE_CHANGED_ATTRIBUTE



UPDATE_WHERE_ATTRIBUTE

```
char *attribute_name;
char *data;
```

UPDATE_WHERE_ATTRIBUTE

UPDATE_CHANGED_ATTRIBUTE

```
/* Usage */
LIST *update_changed_attribute_list(
    DICTIONARY *post_dictionary,
    DICTIONARY *file_dictionary,
    LIST *folder_attribute_list,
    int row );

UPDATE_CHANGED_ATTRIBUTE *update_changed_attribute(
    /* Sets preupdate_$attribute_name for trigger */
    DICTIONARY *post_dictionary,
    DICTIONARY *file_dictionary,
    FOLDER_ATTRIBUTE *folder_attribute,
    int row );

/* Input */
FOLDER_ATTRIBUTE *folder_attribute;

/* Process */
UPDATE_CHANGED_ATTRIBUTE_DATA *update_changed_attribute_data(
    this->post_dictionary,
    this->file_dictionary,
    this->folder_attribute->attribute->attribute_name,
    this->folder_attribute->attribute->attribute_datatype,
    this->folder_attribute->primary_key_index,
    this->row );

char *update_changed_attribute_preupdate_label(
    char *attribute_name,
    int row );

dictionary_set(
    this->post_dictionary,
    this->update_changed_attribute_preupdate_label,
    this->update_changed_attribute_data->old_data );

LIST *update_primary_changed_attribute_list(
    update_changed_attribute_list() );
```

FOLDER_ATTRIBUTE

```
FOLDER *folder;
ATTRIBUTE *attribute;
int primary_key_index;
```

FOLDER

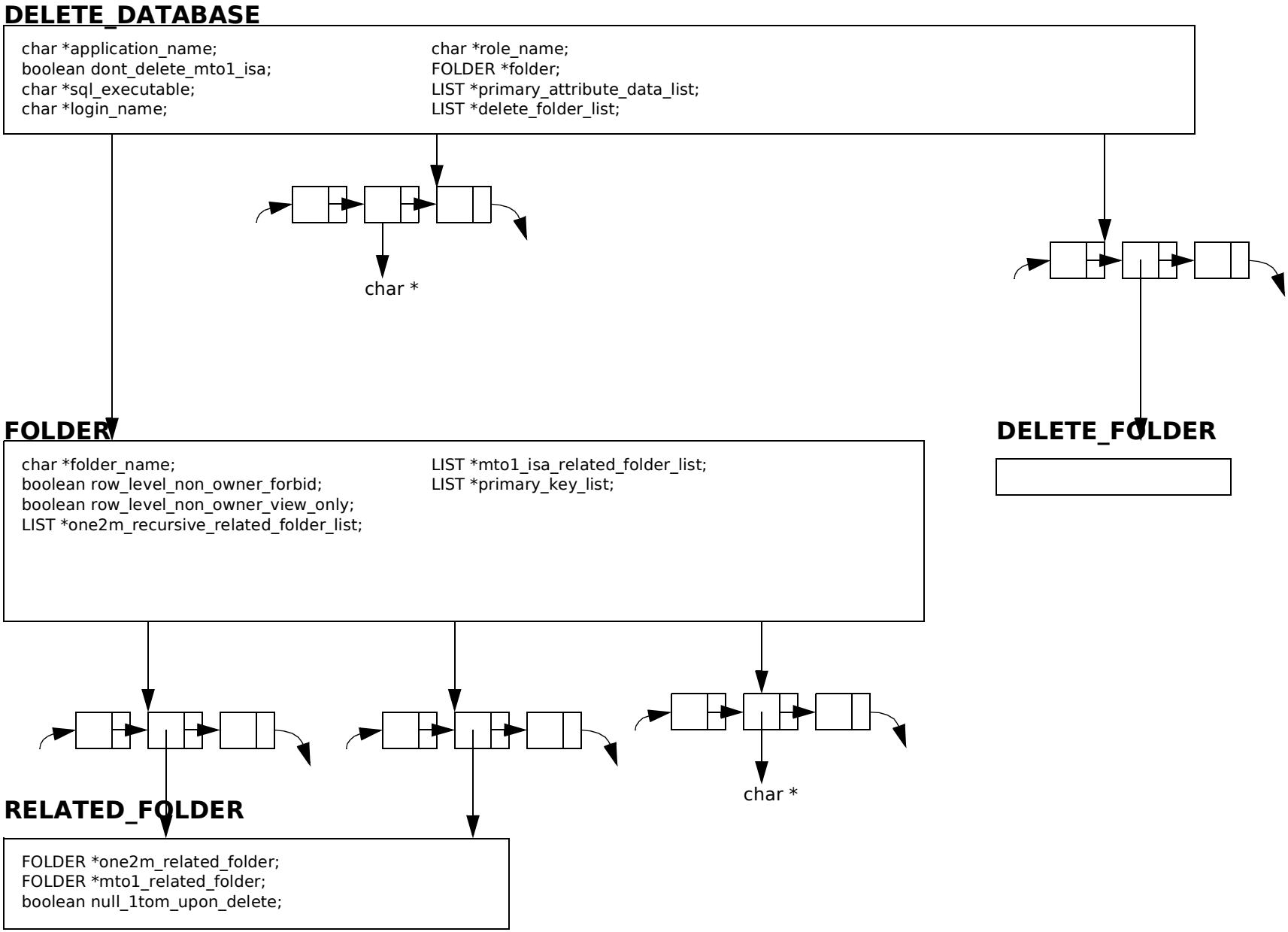
```
char *folder_name;
```

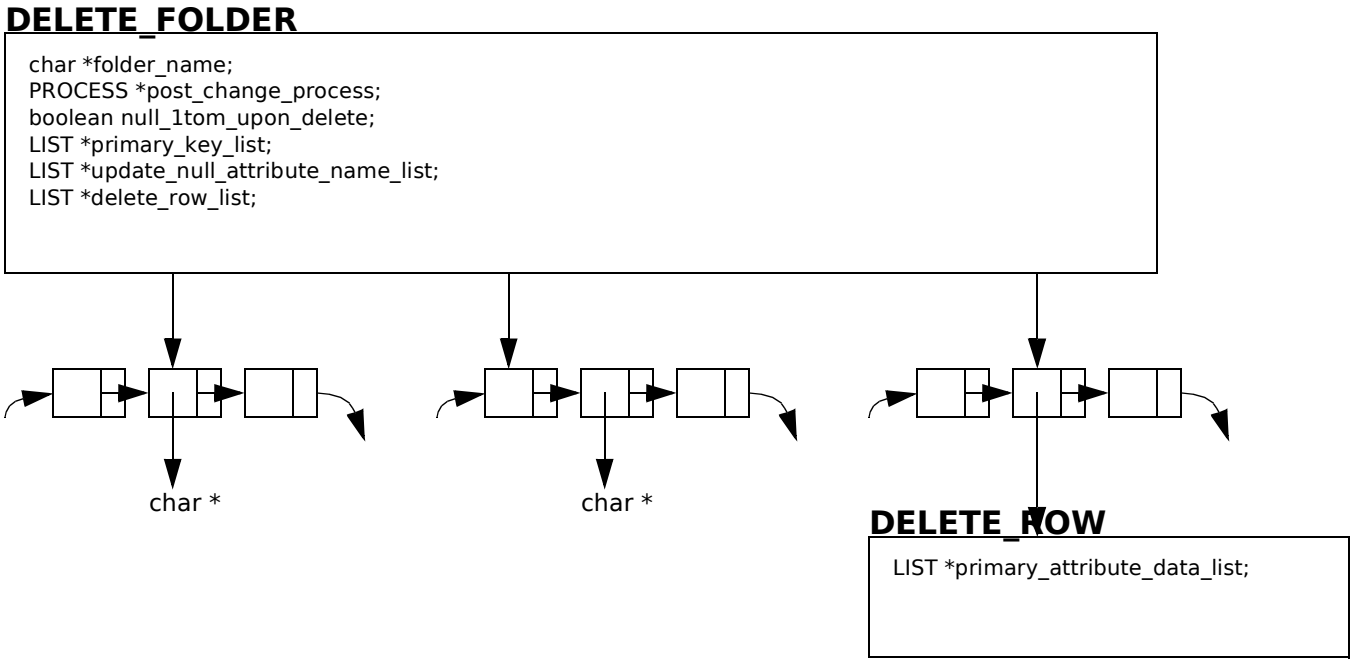
ATTRIBUTE

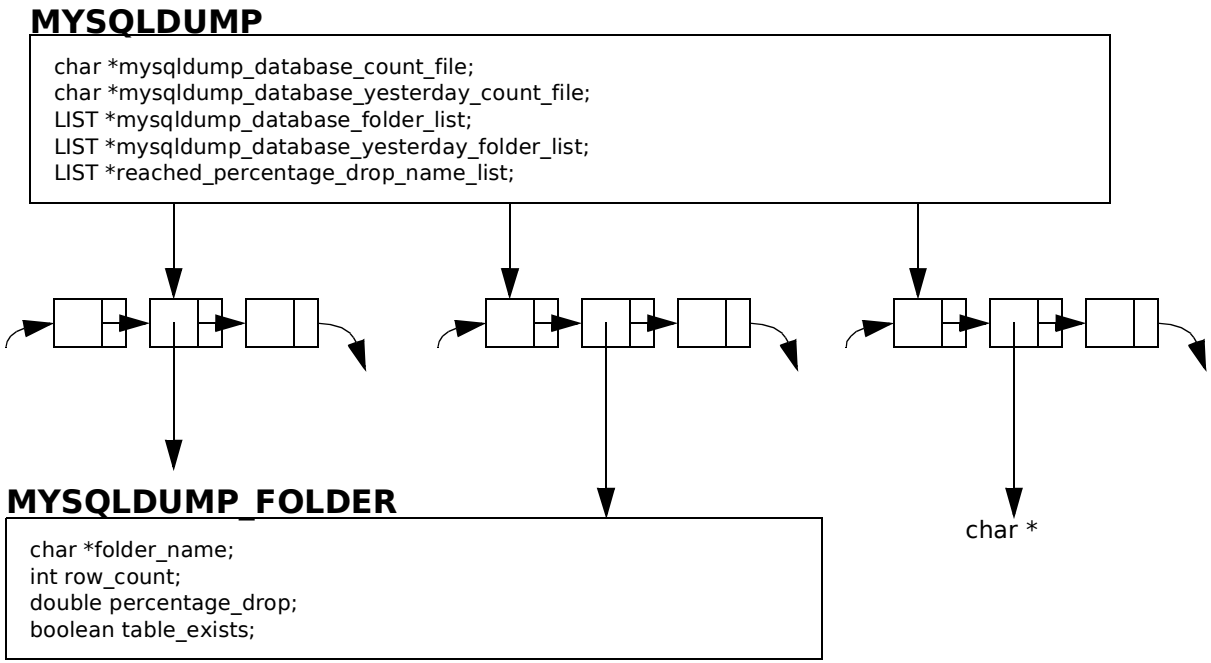
```
char *attribute_name;
char *attribute_datatype;
```

UPDATE_CHANGED_ATTRIBUTE_DATA

```
/* Usage */
UPDATE_CHANGED_ATTRIBUTE_DATA *
    update_changed_attribute_data(
        DICTIONARY *post_dictionary,
        DICTIONARY *file_dictionary,
        char *attribute_name,
        char *attribute_datatype,
        int primary_key_index,
        int row );

/* Process */
char *key =
    update_dictionary_key(
        attribute_name,
        row );

char *old_data =
    dictionary_fetch(
        key,
        file_dictionary );

char *new_data =
    dictionary_fetch(
        key,
        post_dictionary );

boolean changed_attribute =
    ( strcasecmp( old_data, new_data ) != 0 );

char *escaped_replaced_new_data =
        security_sql_injection_escape(
            security_replace_special_characters(
                string_trim_number_characters(
                    new_data,
                    attribute_datatype ) ) );
```


DELETE_DATABASE
char *application_name;
boolean dont_delete_mto1_isa;
char *sql_executable;
char *login_name;
char *role_name;
FOLDER *folder;
LIST *primary_attribute_data_list;
LIST *delete_folder_list;
char *
FOLDER
char *folder_name;
boolean row_level_non_owner_forbid;
boolean row_level_non_owner_view_only;
LIST *one2m_recursive_related_folder_list;
LIST *mto1_isa_related_folder_list;
LIST *primary_key_list;
DELETE_FOLDER
char *
RELATED_FOLDER
FOLDER *one2m_related_folder;
FOLDER *mto1_related_folder;
boolean null_1tom_upon_delete;


DELETE_FOLDER
char *folder_name;
PROCESS *post_change_process;
boolean null_1tom_upon_delete;
LIST *primary_key_list;
LIST *update_null_attribute_name_list;
LIST *delete_row_list;
char *
char *
DELETE_ROW
LIST *primary_attribute_data_list;



STRING

```
boolean string_strcpy(
    char *destination,
    char *source,
    int buffer_size );

boolean string_strcmp(
    char *string1,
    char *string2 );

char *string_escape_character(
    char *destination,
    char *data,
    char character );

char *string_escape_character_array(
    char *destination,
    char *data,
    char *character_array );

/* Position is 1 based */
int string_character_position(
    char *data,
    char character );

char *string_trim_character(
    char *data,
    char character );

char *string_trim_character_array(
    char *data,
    char *character_array );

char *string_trim_number_characters(
    char *number,
    char *attribute_datatype )
{
    return string_trim_character_array(
        number,
        ",$" );
}
```

SECURITY

```
enum password_function password_function; { no_encryption
                                            old_password_function
                                            regular_password_function
                                            sha2_function

enum password_function
    security_database_password_function(
        char *database_password );

char *security_encrypted_password(
    char *password_sql_injection_escape,
    enum password_function password_function );

char *security_mysql_version( void );

enum password_function
    security_mysql_version_password_function(
        security_mysql_version() );

char *security_encryption_select_clause(
    enum password_function password_function,
    char *password );

boolean security_password_match(
    database_password,
    security_encrypted_password(
        security_sql_injection_escape(
            typed_in_password ),
        security_database_password_function() );

boolean security_password_encrypted(
    char *database_password );

char *security_replace_special_characters(
    char *data );

char *security_sql_injection_escape(
    char *data )
{
    char destination[ QUERY_WHERE_BUFFER ];

    return strdup(
        string_escape_character_array(
            destination,
            data,
            “‘’$;%$=” ) );
}
```

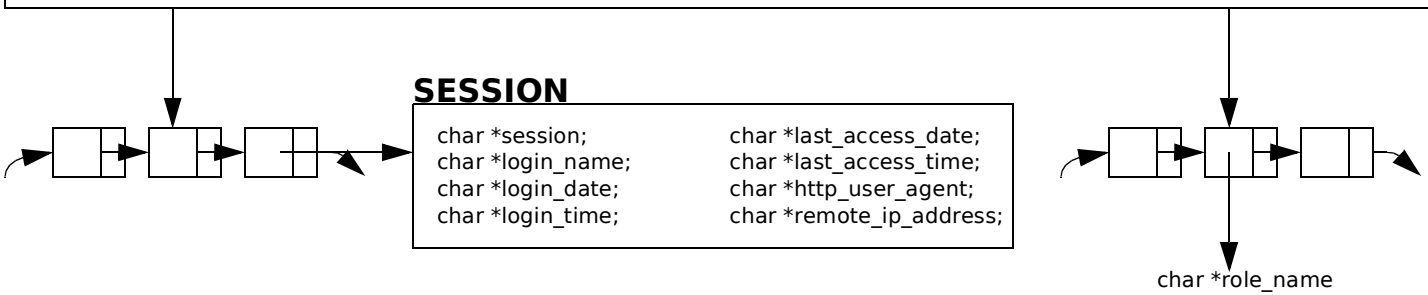
SECURITY_ENTITY

```
/* Usage */
SECURITY_ENTITY *security_entity(
    char *login_name,
    boolean folder->non_owner_forbid,
    boolean role->override_row_restrictions );

/* Input */
char *login_name;
boolean non_owner_forbid;
boolean override_row_restrictions;

/* Process */
char *login_name_only;
char *full_name_only;
char *street_address_only;

/* Returns null if no ENTITY->login_name */
char *security_login_name_full_name_only(
    &this->street_address_only,
    this->login_name );

/* Returns full_name_only */
char *security_entity_fetch(
    &this->street_address_only,
    this->login_name );

char *security_entity_where_clause(
    char *where_clause,
    SECURITY_ENTITY *security_entity );
```

APPASERVER Data Structure

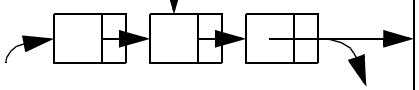
APPASERVER_USER

```
/* Usage */
APPASERVER_USER *appaserver_user_fetch(
    char *login_name,
    boolean fetch_role_name_list,
    boolean fetch_session_list );

/* Input */
char *login_name;
char *full_name;
char *typed_in_password;
char *database_password;
char *user_date_format;

/* Process */
APPASERVER_USER *appaserver_user_calloc( void );

APPASERVER_USER *appaserver_user_parse(
    char *input,
    boolean fetch_role_name_list,
    boolean fetch_session_list );

LIST *appaserver_user_role_list(
    char *login_name );

LIST *appaserver_user_session_list(
    char *login_name );

FILE *appaserver_user_update_open( void );

void appaserver_user_update(
    appaserver_user_update_open(),
    security_encrypted_password(
        security_sql_injection_escape(
            database_password ),
        security_mysql_version_password_function(
            security_mysql_version() ) ),
    login_name );
```



PAIR_ONE2M

```
char *many_folder_name;
char *one_folder_name;

PAIR_ONE2M *pair_one2m_prompt_form_new(
    char *one_folder_name,
    char *keystrokes_save_function );

LIST *pair_one2m_prompt_form_folder_list(
    char *keystrokes_save_function,
    LIST *relation_one2m_pair_relation_list() );

char *pair_one2m_prompt_element_name(
    char *PAIR_ONE2M_PREFIX,
    char *folder_label );

boolean pair_one2m_participating(
    DICTIONARY *pair_one2m_dictionaryy );

PAIR_ONE2M *pair_one2m_post_new(
    DICTIONARY *pair_one2m_dictionary );

boolean pair_one2m_duplicate(
    char *MYSQL_DUPLICATE_ERROR_MESSAGE_KEY,
    char *message );

void pair_one2m_duplicate_set(
    DICTIONARY *pair_one2m_dictionary,
    char *PAIR_ONE2M_DUPLICATE_KEY );

boolean pair_one2m_inserted_duplicate(
    char *PAIR_ONE2M_DUPLICATE_KEY,
    DICTIONARY *pair_one2m_dictionary );

LIST *pair_one2m_fulfilled_folder_name_list(
    char *PAIR_ONE2M_FULFILLED_LIST_LABEL,
    DICTIONARY *pair_one2m_dictionary );

char *pair_one2m_folder_name(
    char *folder_label,
    DICTIONARY *pair_one2m_dictionary );

void pair_one2m_folder_set(
    DICTIONARY *pair_one2m_dictionary,
    char *folder_label,
    char *folder_name );

char *pair_one2m_next_folder_name(
    LIST *pair_one2m_fulfilled_folder_name_list(),
    LIST *relation_one2m_pair_relation_list(),
    char *one_folder_name,
    char *many_folder_name );

DICTIONARY  *pair_one2m_fulfilled_dictionary(
    DICTIONARY *pair_one2m_dictionary,
    char *PAIR_ONE2M_FULFILLED_LIST_LABEL,
    LIST *pair_one2m_fulfilled_folder_name_list );

/* Insert form */
PAIR_ONE2M *pair_one2m_insert_form_new(
    DICTIONARY *pair_one2m_dictionary );
```



PAIR_ONE2M_FOLDER

```
/* Usage */
pair_one2m_folder_new(
    char *many_folder_name );

char *many_folder_name;

char *pair_folder_element_copy_function(
    prompt_many_element_name,
    many_folder_name,
    pair_one2m_prompt_form_element_name(),
    folder_name );

char *pair_folder_button_string(
    many_folder_name,
    keystrokes_save_function,
    pair_folder_element_copy_function() );
```

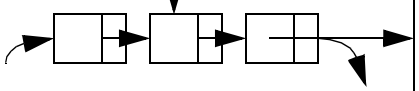
VERTICAL_NEW_BUTTON

```
char *many_folder_name;
char *one_folder_name;

VERTICAL_NEW_BUTTON *vertical_new_button_calloc(
    void );

char *vertical_new_button_one_element_name(
    char *VERTICAL_NEW_BUTTON_ONE_PREFIX,
    char *one_folder_name );

char *vertical_new_button_many_element_name(
    char *VERTICAL_NEW_BUTTON_MANY_LABEL );

ELEMENT_APPASERVER *vertical_new_button_element(
    char *one_folder_name,
    LIST *role_folder_insert_list,
    char *form_name,
    char *VERTICAL_NEW_BUTTON_ONE_PREFIX,
    char *VERTICAL_NEW_BUTTON_DISPLAY,
    char *onclick_keystrokes_save_string );

char *vertical_new_button_dictionary_one_folder_name(
    char *VERTICAL_NEW_BUTTON_ONE_PREFIX,
    DICTIONARY *non_prefixed_dictionary );

char *vertical_new_button_dictionary_folder_name(
    char *hidden_label,
    DICTIONARY *non_prefixed_dictionary );

void vertical_new_button_dictionary_set(
    DICTIONARY *non_prefixed_dictionary,
    char *hidden_label,
    char *folder_name );
```

INSERT_DATABASE

```
char *application_name;
char *session;
char *folder_name;
LIST *primary_key_list;
LIST *attribute_name_list;
LIST *data_list;
DICTIONARY *row_dictionary;
LIST *ignore_insert_attribute_name_list;
boolean insert_row_zero_only;
boolean dont_remove_tmp_file;
int rows_inserted;
char *post_change_process;
char *login_name;
char *tmp_file_name;
FILE *tmp_file;
LIST *attribute_list;
```

```
LIST *attribute_data_list =
    insert_database_attribute_data_list(
        row_dictionary,
        row,
        ignore_insert_attribute_name_list,
        primary_key_list,
        attribute_name_list,
        mto1_related_folder_list,
        common_non_primary_key_list,
        application_name,
        attribute_list );

LIST *insert_attribute_name_list =
    insert_database_attribute_name_list(
        insert_database_attribute_data_list() );
```

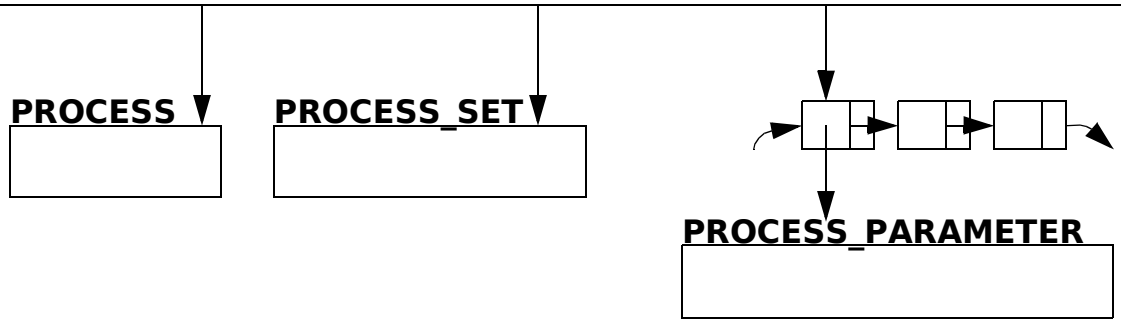


INSERT_DATABASE_ATTRIBUTE_DATA

```
/* Usage */
INSERT_DATABASE_ATTRIBUTE_DATA *
    insert_database_attribute_data_new(
        char *attribute_name,
        char *security_sql_injection_escape(
            security_replace_special_characters(
                string_trim_number_characters(
                    char *data,
                    char *attribute_datatype ) ) ) );

char *attribute_name;
char *escaped_replaced_data;
```

PROCESS_STRUCTURE

```
/* Usage */
PROCESS_STRUCTURE *process_structure_fetch(
    char *process_or_process_set_name,
    char *role_name,
    char *login_name,
    boolean check_executable_inside_filesystem,
    boolean is_preprompt,
    DICTIONARY *preprompt_dictionary );

/* Input */
char *process_or_process_set_name;
char *role_name;
boolean check_executable_inside_filesystem;
DICTIONARY *preprompt_dictionary;

/* Process */
boolean process_structure_exists_preprompt(
    char *process_or_process_set_name );

boolean process_structure_is_set(
    char *process_or_process_set_name );

PROCESS *process_fetch(
    this->process_or_process_set_name,
    this->role_name,
    this->login_name,
    this->check_executable_inside_filesystem );

LIST *process_parameter_list =
    process_parameter_system_list(
        process_parameter_system_string(
            process_primary_where(
                this->process_or_process_set_name ) ),
        this->preprompt_dictionary,
        this->login_name );

PROCESS_SET *process_set_fetch(
    this->process_or_process_set_name,
    this->role_name,
    this->check_executable_inside_filesystem );

LIST *process_parameter_list =
    process_parameter_system_list(
        process_set_parameter_system_string(
            process_set_primary_where(
                this->process_or_process_set_name ) ),
        this->preprompt_dictionary,
        this->login_name );
```

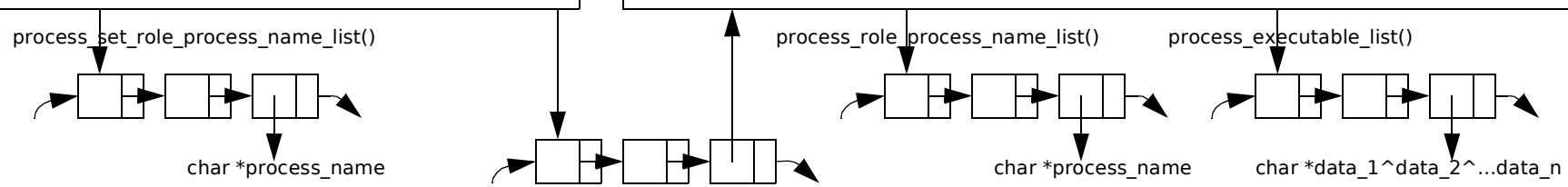


PROCESS_SET

```
/* Usage */
PROCESS_SET *process_set_fetch(
    char *process_set_name,
    char *role_name,
    char *login_name,
    boolean check_executable_inside_filesystem );

/* Input */
char *process_set_name;
char *role_name;
boolean check_executable_inside_filesystem;
char *notepad;
char *html_help_file_anchor;
char *post_change_javascript;
char *prompt_display_text;
char *process_group;
char *preprompt_help_text;
boolean prompt_display_bottom;

/* Process */
PROCESS_SET *process_set_parse(
    char *input,
    char *role_name,
    boolean check_executable_inside_filesystem );

char *process_set_primary_where(
    char *process_set_name );

LIST *process_set_role_process_name_list(
    process_set_primary_where(),
    this->role_name );

char *process_set_process_where(
    process_set_role_process_name_list() );

LIST *process_list =
    process_system_list(
        process_system_string(
            process_set_process_name_where() ),
        this->check_executable_inside_filesystem );
```

PROCESS

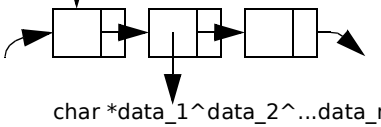
```
/* Usage */
PROCESS *process_fetch(
    process_name,
    role_name,
    login_name,
    check_executable_inside_filesystem );

/* Input */
char *process_name;
char *role_name;
boolean check_executable_inside_filesystem;
char *command_line;
char *notepad;
char *html_help_file_anchor;
int execution_count;
char *post_change_javascript;
char *process_set_display;
char *process_group;
char *preprompt_help_text;

/* Process */
char *process_primary_where(
    char *process_name );

char *process_system_string(
    char *where );

LIST *process_system_list(
    process_system_string(),
    check_executable_inside_filesystem );

LIST *process_role_process_name_list(
    role_primary_where(
        this->role_name ) );

boolean process_name_exists(
    process_name,
    process_role_process_name_list() );

LIST *process_executable_list(
        char *command_line );
```



PROCESS_PARAMETER

```
/*  Usage */
LIST *process_parameter_system_list(
    char *system_string,
    DICTIONARY *preprompt_dictionary,
    char *login_name,
    char *role_name );

/* Input */
char *process_or_set_name;
DICTIONARY *preprompt_dictionary;
char *login_name;
char *role_name,
char *folder_name;
char *attribute_name;
char *drop_down_prompt_name;
char *prompt_name;
int display_order;
boolean drop_down_multi_select;
boolean preprompt;
char *populate_drop_down_process_name;
char *populate_helper_process_name;

/* Process */
char *process_parameter_system_string(
    char *where );

char *process_set_parameter_system_string(
    char *where );

PROCESS_PARAMETER *process_parameter_parse(
    char *input,
    DICTIONARY *preprompt_dictionary,
    char *login_name );

LIST *process_parameter_executable_list(
        char *command_line );

LIST *process_parameter_primary_delimited_list(
    this->preprompt_dictionary,
    this->login_name,
    this->role_name,
    this->folder_name,
    this->populate_drop_down_process_name );

LIST *process_parameter_evaluate_list(
    process_fetch(
        populate_drop_down_process_name,
        1 /* check_executable... */ )->
            command_line,
    preprompt_dictionary,
    login_name,
    role_name,
    folder_name );

void process_parameter_command_line_replace(
    &command_line,
    preprompt_dictionary,
    login_name,
    role_name,
    folder_name,
    environment_application_name() );

void process_parameter_search_replace_where(
    command_line,
    preprompt_dictionary,
    login_name,
    role_name,
    folder_name );

ATTRIBUTE *attribute =
    attribute_fetch(
        this->attribute_name );

DROP_DOWN_PROMPT *drop_down_prompt =
    drop_down_prompt_fetch(
        this->drop_down_prompt_name );

PROMPT *prompt =
    prompt_fetch(
        this->prompt_name );
```

process_parameter_primary_delimited_list()



char *data_1^data_2^...data_n

ATTRIBUTE

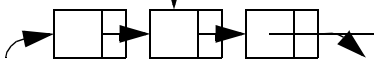
```
char *attribute_name;
```

DROP_DOWN_PROMPT

```
/* Usage */
DROP_DOWN_PROMPT *drop_down_prompt_fetch(
    char *drop_down_prompt_name );

char *drop_down_prompt_name;
char *hint_message;
char *optional_display;

LIST *drop_down_prompt_data_list =
    drop_down_prompt_data_system_list(
        drop_down_prompt_data_system_string(
            drop_down_prompt_primary_where(
                this->drop_down_prompt_name ) ) );
```

PROMPT

```
/* Usage */
PROMPT *prompt_fetch(
        char *prompt_name );

/* Input */
char *prompt_name;
int input_width;
char *hint_message;
boolean upload_filename;
boolean date;
```



DROP_DOWN_PROMPT_DATA

```
char *drop_down_prompt_name;
char *drop_down_prompt_data;
int display_order;
```

PROCESS_GENERIC (1)

```
/* Usage */
PROCESS_GENERIC *process_generic_fetch(
    process_set_name,
    process_name,
    login_name,
    output_medium_string,
    post_dictionary );

char *process_set_name;
char *login_name;
char *output_medium_string;
DICTIONARY *post_dictionary;

/* Process */
char *process_name =
    process_generic_process_name(
        this->process_set_name,
        post_dictionary );

char *value_folder_name =
    process_generic_value_folder_name(
        this->process_name,
        this->process_set_name );

PROCESS_GENERIC_VALUE_FOLDER *value_folder =
    process_generic_value_folder_fetch(
        this->value_folder_name );

PROCESS_GENERIC_PARAMETER *parameter =
    process_generic_parameter_parse(
        this->output_medium_string,
        this->post_dictionary,
        this->value_folder->datatype->aggregation_sum,
        this->value_folder->primary_key_list,
        this->value_folder->date_attribute_name,
        this->value_folder->time_attribute_name );

char *process_generic_select(
    this->value_folder->primary_key_list,
    this->parameter->value_attribute_name );

char *process_generic_date_where(
    this->value_folder->date_attribute_name,
    this->parameter->begin_date,
    this->parameter->end_date );

char *process_generic_where(
    this->value_folder->foreign_folder->foreign_attribute_name_list,
    this->value_folder->foreign_folder->foreign_attribute_data_list,
    process_generic_date_where() );

char *process_generic_system_string(
    process_generic_select(),
    this->value_folder_name,
    process_generic_where(),
    this->parameter->aggregate_level,
    this->parameter->aggregate_statistic,
    this->parameter->accumulate,
    this->parameter->date_piece,
    this->parameter->time_piece,
    this->parameter->value_piece,
    this->parameter->end_date,
    this->value_folder->datatype->unit /* datatype_unit */,
    this->value_folder->foreign_folder->unit /* foreign_folder_unit */,
    this->parameter->units_converted );

char *process_generic_heading(
    this->value_folder->primary_key_list,
    this->value_folder->value_attribute_name,
    this->value_folder->datatype->unit /* datatype_unit */,
    this->value_folder->foreign_folder->unit /* foreign_folder_unit */,
    this->parameter->units_converted,
    this->parameter->aggregate_level,
    this->parameter->aggregate_statistic,
    this->parameter->accumulate );

char *process_generic_subtitle(
    this->value_folder_name,
    this->parameter->begin_date,
    this->parameter->end_date,
    this->parameter->aggregate_level,
    this->parameter->aggregate_statistic
    additional_message );
```

PROCESS_GENERIC_PARAMETER

PROCESS_GENERIC_VALUE_FOLDER

PROCESS_GENERIC (2)

```
char *units_label(
    units,
    units_converted,
    enum aggregate_statistic );

enum date_convert_format process_generic_date_convert_format(
    login_name );

boolean process_generic_no_cycle_color(
    role_name );
```

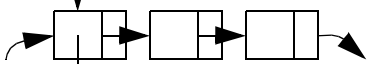
PROCESS_GENERIC_PARAMETER

```
/* Attributes */
char *output_medium_string;
enum process_generic_medium output_medium;
char *begin_date;
char *end_date;
enum aggregate_level aggregate_level;
enum aggregate_statistic aggregate_statistic;
boolean accumulate;
char *email_address;
char *units_converted;
pidt process_id;
int date_piece;
int time_piece;
int value_piece;

/* Usage */
PROCESS_GENERIC_PARAMETER *process_generic_parameter_parse(
    process_generic->output_medium_string,
    process_generic_>post_dictionary,
    process_generic->value_folder->datatype->aggregation_sum,
    value_folder_primary_key_list,
    value_folder_date_attribute_name,
    value_folder_time_attribute_name );
```

{ text_file, output_medium_stdout, table, spreadsheet, gracechart, output_medium_unknown

PROCESS_GENERIC_VALUE_FOLDER

```
/* Usage */
PROCESS_GENERIC_VALUE_FOLDER *value_folder =
    process_generic_value_folder_fetch(
        process_generic->value_folder_name,
        process_generic->post_dictionary );

/* Input */
char *value_folder_name;
DICTIONARY *post_dictionary;

/* Process */
PROCESS_GENERIC_VALUE_FOLDER *value_folder_parse(
    input );

char *datatype_folder_name;
char *foreign_folder_name;
char *datatype_attribute_name;
char *date_attribute_name;
char *time_attribute_name;
char *value_attribute_name;
boolean exists_aggregation_sum;
boolean exists_bar_graph;
boolean exists_scale_graph_zero;
boolean datatype_exists_unit;
boolean foreign_exists_unit;

LIST *primary_key_list =
    folder_primary_key_list_fetch(
        this->value_folder_name );

PROCESS_GENERIC_DATATYPE *datatype =
    process_generic_datatype_fetch(
        this->datatype_folder_name,
        this->datatype_attribute_name,
        this->exists_aggregation_sum,
        this->exists_bar_graph,
        this->exists_scale_graph_zero,
        this->datatype_exists_units,
        this->post_dictionary );

PROCESS_GENERIC_FOREIGN_FOLDER *foreign_folder =
    process_generic_foreign_folder_fetch(
        this->foreign_folder_name,
        this->foreign_exists_unit,
        this->post_dictionary );

LIST *process_generic_datatype_compare_list(
    process_generic->post_dictionary,
    this->exists_aggregation_sum,
    this->exists_bar_graph,
    this->exists_scale_graph_zero,
    this->datatype_exists_unit );
```



PROCESS_GENERIC_FOREIGN_FOLDER

PROCESS_GENERIC_DATATYPE

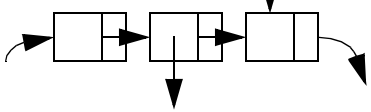
char *

PROCESS_GENERIC_FOREIGN_FOLDER

```
char *foreign_folder_name;
FOLDER *foreign_folder;

/* Usage */
PROCESS_GENERIC_FOREIGN_FOLDER *process_generic_foreign_folder_fetch(
    process_generic->value_folder->foreign_folder_name,
    process_generic->post_dictionary,
    process_generic->value_folder->foreign_exists_unit );

LIST *foreign_attribute_name_list =
    foreign_folder->primary_key_list;

LIST *foreign_attribute_data_list =
    dictionary_key_list_fetch(
        this->foreign_attribute_name_list
            /* key_list */,
        this->post_dictionary );

char *unit = (foreign_exists_unit) ? list_last( foreign_attribute_data_list ) : (char *)0;
```

FOLDER

```
char *folder_name;
LIST *primary_key_list
```

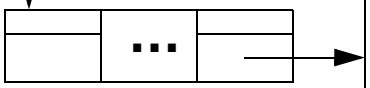


PROCESS_GENERIC_DATATYPE

```
/* Usage */
PROCESS_GENERIC_DATATYPE *process_generic_datatype_fetch(
    process_generic_value_folder->datatype_folder_name,
    process_generic_value_folder->datatype_attribute_name,
    process_generic_value_folder->exists_aggregation_sum,
    process_generic_value_folder->exists_bar_graph,
    process_generic_value_folder->exists_scale_graph_zero,
    process_generic_value_folder->datatype_exists_units,
    process_generic_value_folder->post_dictionary );

/* Input */
char *datatype_folder_name;
char *datatype_attribute_name;
boolean exists_aggregation_sum;
boolean exists_bar_graph;
boolean exists_scale_graph_zero;
boolean datatype_exists_units;
DICTIONARY *post_dictionary;

/* Process */
char *datatype_name;
boolean aggregation_sum;
boolean bar_graph;
boolean scale_graph_zero;
char *unit;
```

```
char *process_generic_datatype_select(
    this->datatype_attribute_name,
    this->exists_aggregation_sum,
    this->exists_bar_graph,
    this->exists_scale_graph_zero,
    this->datatype_exists_unit );

char *datatype_name =
    dictionary_fetch(
        this->datatype_attribute_name,
        this->post_dictionary );

char *process_generic_datatype_where(
    this->datatype_attribute_name,
    this->datatype_name );

char *process_generic_datatype_system_string(
    process_generic_datatype_select(),
    process_generic_datatype->datatype_folder_name,
    process_generic_datatype_where() );

PROCESS_GENERIC_DATATYPE *process_generic_datatype_parse(
    string_pipe_fetch(
        process_generic_datatype_system_string() )
            /* input_buffer */ );
```

key = date or
key = date^time



PROCESS_GENERIC_VALUE

```
char *date;
char *time;
char *unit;
double value;
boolean is_null;
double accumulate
```

```
PROCESS_GENERIC_VALUE *process_generic_value_parse(
    double *accumulate_value,
    char *input,
    char *time_attribute,
    boolean unit_value_folder,
    boolean accumulate_boolean,
    enum aggregate_level );
```